

2. Funktionale Programmiersprachen

- 1 Konzepte von Programmiersprachen
- 2 **Funktionale Programmiersprachen**
 - a. Grundlagen der funktionalen Programmierung
 - b. Einführung in Clojure
 - c. **Funktionale Programmierung in Java 8**
- 3 Sprachverarbeitung
- 4 Logikorientierte Programmiersprachen

Iteration in Java

- Meist verwendete Iteration in Java

```
int[] myArray = new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9};  
  
for( int i = 0; i < myArray.length; i++ ) {  
    System.out.println("i = " + i);  
}
```

- Iteration mit erweiterter For-Schleife
 - Intern wird aber nicht parallel, sondern seriell gearbeitet (Iterator)
 - Trotzdem besserer Programmierstil

```
int[] myArray = new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9};  
  
for( int j: myArray ) {  
    System.out.println("j = " + j);  
}
```

Einführung in Java 8

- Sortieren von Strings in Java 7
 - Verwendung von anonymous inner class

```
// Sortierung mit Comparator
final List<String> names
    = Arrays.asList("Andy", "Mike", "Max", "Steve");

Collections.sort(names, new Comparator<String>() {
    @Override
    public int compare(final String str1, final String str2) {
        return Integer.compare(str1.length(), str2.length());
    }
});

// Ausgabe
names.forEach(it -> System.out.print(it + ", "));
// Max, Andy, Mike, Steve
```

Codebeispiele aus: M. Inden: Java 8

Lambdas in Java 8

- Lambdas in Java 8 haben
 - keinen Namen
 - keine explizite Angabe eines Rückgabetyps
 - keine Deklaration von Exceptions

```
(int x, int y) -> { return x + y; }
```

```
(long x) -> { return x * 2; }
```

```
() -> { String msg = "Lambda";  
        System.out.println("Hello " + msg); }
```

Functional Interfaces in Java 8

- Type Inference
 - Verzicht auf Typangaben für Parameter
- Kurzform
 - Für folgende Ausdrücke

```
(int x, int y) -> { return x + y; }  
(long x) -> { return x * 2; }
```

- ergibt sich folgende Kurzform

```
(x, y) -> x + y  
x -> x * 2
```

Functional Interfaces in Java 8

```
// Compile-Error: incompatible types:  
// Object is not a functional interface  
  
Object greeter = () -> { System.out.println("Hi Lambda"); };
```

- Functional Interface
 - Neuer Art von Typ mit genau einer abstrakten Methode
 - SAM-Typ (**S**ingle **A**bstract **M**ethod)
 - Beispiele: Runnable, Comparator

```
@FunctionalInterface  
public interface Runnable  
{  
    public abstract void run();  
}
```

Functional Interfaces in Java 8

- Einige Ein- und Zwei-Argument-Methoden (aus java.util.function)

Typ	Schnittstelle	Operation
Konsument	Consumer<T>	void accept(T t)
	BiConsumer<T,U>	void accept(T t, U u)
Funktion	Function<T,R>	R apply(T t)
	BiFunction<T,U,R>	R apply(T t, U u)
Prädikat	Predicate<T>	boolean test(T t)
	BiPredicate<T,U>	boolean test(T t, U u)

- Beispiel

```
BiFunction<Integer, Integer, Integer> pl = (a,b) -> a+b ;  
System.out.println( pl.apply( 1,2 ) );
```

Lambdas in Java 8

- Lambdas zur Sortierung von Namen
 - Zwei Varianten

```
public static void main(final String[] args) {
    final List<String> names
        = Arrays.asList("Andy", "Michael", "Max", "Stefan");

    // Lambda als Methodenparameter
    Collections.sort(names,
        (str1, str2) -> Integer.compare(str1.length(), str2.length()));

    // Alternative mit Lambda als Rückgabe einer Methode
    names.sort(compareByLength());
}

public static Comparator<String> compareByLength() {
    return (str1, str2)
        -> Integer.compare(str1.length(), str2.length());
}
```


Lambdas in Java 8

- Möglichst optimale Programmierung in Java 8

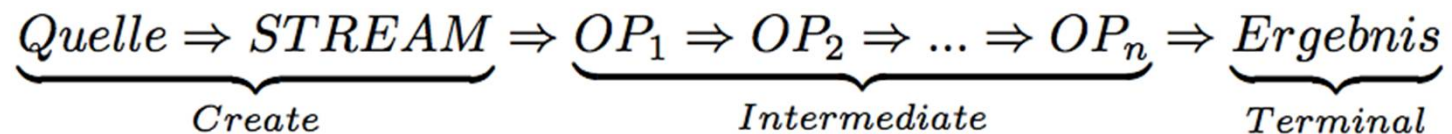
```
final List<String> names = Arrays.asList("Andy", "Mike", "Max", "Steve");  
names.sort( (str1, str2) -> Integer.compare(str1.length(), str2.length()) );  
names.forEach(it -> System.out.print(it + ", "));
```

- Vergleich zu Clojure

```
(sort #(< (.length %1) (.length %2)) ["Andy", "Mike", "Max", "Steve"])
```

Streams in Java 8

- Streams
 - Folge von Bearbeitungsschritten
 - Interface `java.util.stream.Stream`

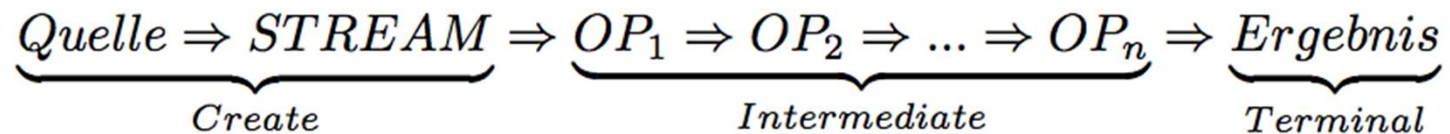


```
List<Integer> lst = Arrays.asList(6, 3, 1, 4, 0, 17, 33);  
lst.stream()  
    .filter(x -> x % 2 == 0)  
    .map(x -> x * x)  
    .forEach(System.out::println);
```

Quelle: M. Inden: Java 8 – Die Neuerungen

Streams in Java 8

- Intermediate Operations – einige Beispiele
 - `filter`
 - `map`
 - `distinct`
 - `sorted`
- Terminal Operations – einige Beispiele
 - `foreach`
 - `reduce`
 - `count`
 - `toArray`



Quelle: M. Inden: Java 8 – Die Neuerungen

Unendliche Streams in Java 8

- Unendliche Streams
 - Unendliche Folge von Werten
 - Methoden `iterate`, `generate`

```
final IntStream iteratingValues = IntStream.iterate(0, x -> x + 1);  
final int[] firstTen = iteratingValues.limit(10).toArray();
```

Quelle: M. Inden: Java 8 – Die Neuerungen